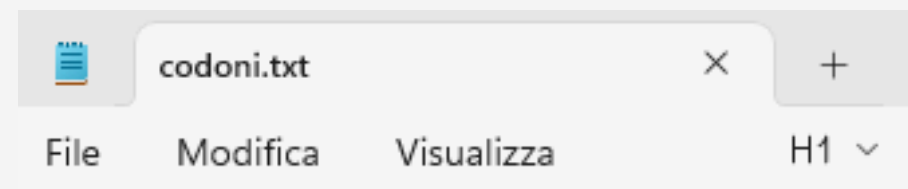


DECIFRARE IL CODICE DELLA VITA

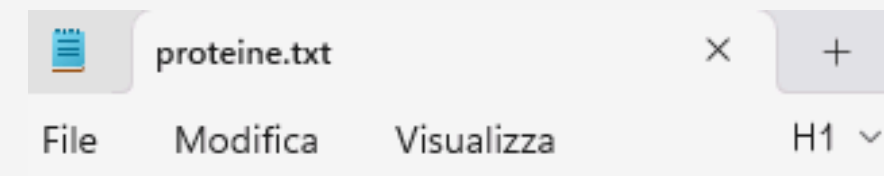
**Introduzione alla bioinformatica per
l'analisi genomica**

Funzionamento del software

Il codice analizza un file di testo sulla quale sono trascritti una serie di codoni e ne produce un altro in cui sono vengono elencati, in ordine, gli amminoacidi da essi codificati



AUGUUUCAUAG

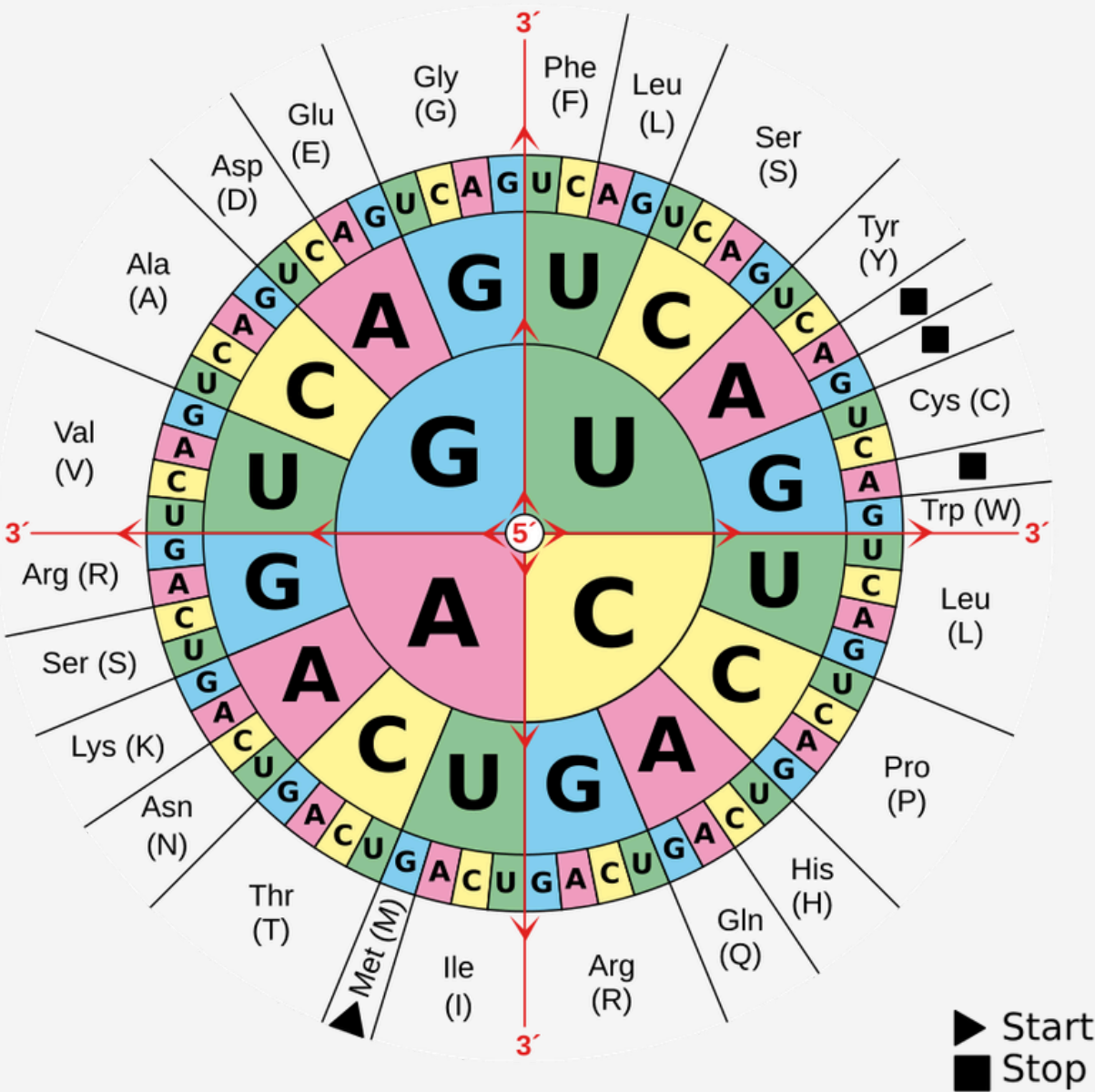


M F S

```
//Phe (F)
codone[U][U][U] = 'F'; codone[U][U][C] = 'F';
//Leu (L)
codone[U][U][A] = 'L'; codone[U][U][G] = 'L';
//Ser (S)
codone[U][C][U] = 'S'; codone[U][C][C] = 'S';
codone[U][C][A] = 'S'; codone[U][C][G] = 'S';
//Uyr (Y)
codone[U][A][U] = 'Y'; codone[U][A][C] = 'Y';
//STOP
codone[U][A][A] = '*'; codone[U][A][G] = '*';
//Cys (C)
codone[U][G][U] = 'C'; codone[U][G][C] = 'C';
// STOP
codone[U][G][A] = '*';
//Urp (W)
codone[U][G][G] = 'W';
//Leu (L)
codone[C][U][U] = 'L'; codone[C][U][C] = 'L';
codone[C][U][A] = 'L'; codone[C][U][G] = 'L';
// Pro (P)
codone[C][C][U] = 'P'; codone[C][C][C] = 'P';
codone[C][C][A] = 'P'; codone[C][C][G] = 'P';
//His (H)
codone[C][A][U] = 'H'; codone[C][A][C] = 'H';
//Gln (Q)
codone[C][A][A] = 'Q'; codone[C][A][G] = 'Q';
//Arg (R)
codone[C][G][U] = 'R'; codone[C][G][C] = 'R';
codone[C][G][A] = 'R'; codone[C][G][G] = 'R';
//Ile (I)
codone[A][U][U] = 'I'; codone[A][U][C] = 'I';
codone[A][U][A] = 'I';
// Met (M)
codone[A][U][G] = 'M';
//Uhr (U)
codone[A][C][U] = 'U'; codone[A][C][C] = 'U';
codone[A][C][A] = 'U'; codone[A][C][G] = 'U';
//Asn (N)
codone[A][A][U] = 'N'; codone[A][A][C] = 'N';
//Lys (K)
codone[A][A][A] = 'K'; codone[A][A][G] = 'K';
//Ser (S)
codone[A][G][U] = 'S'; codone[A][G][C] = 'S';
//Arg (R)
codone[A][G][A] = 'R'; codone[A][G][G] = 'R';
//Val (V)
codone[G][U][U] = 'V'; codone[G][U][C] = 'V';
codone[G][U][A] = 'V'; codone[G][U][G] = 'V';
//Ala (A)
codone[G][C][U] = 'A'; codone[G][C][C] = 'A';
codone[G][C][A] = 'A'; codone[G][C][G] = 'A';
//Asp (D)
codone[G][A][U] = 'D'; codone[G][A][C] = 'D';
//Glu (E)
codone[G][A][A] = 'E'; codone[G][A][G] = 'E';
//Gly (G)
codone[G][G][U] = 'G'; codone[G][G][C] = 'G';
codone[G][G][A] = 'G'; codone[G][G][G] = 'G';
```

Utilizzando le matrici tridimensionali vengono inizializzati tutti i codoni e le proteine da essi codificate basandosi sulla tabella del codice genetico

Si nota che ogni proteina, ad eccezione della metionina, è legata a diversi codoni. Questo rende il codice genetico sia ridondante sia univoco

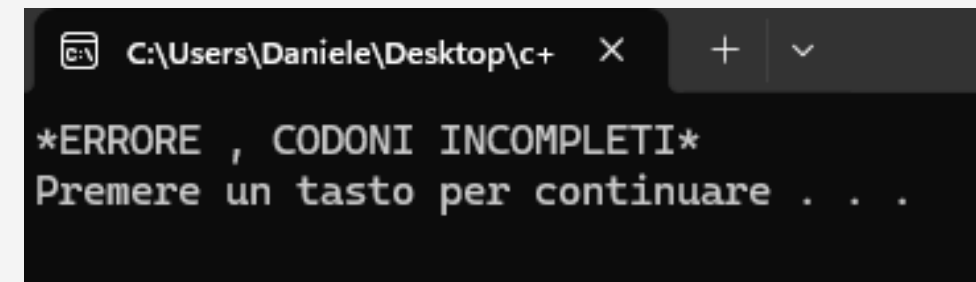


```

ifstream fin("codoni.txt");
ofstream fout("proteine.txt");
char x; bool stop=false; bool errore=false;
int i=0;
while (fin.get(x)&&!stop)
{
    if(x=='A' || x=='C' || x=='G' || x=='U')
    {
        i++;
    }
    else if(x=='\n' || x==' '){/*ignora*/}
    else{stop=true;errore=true;}
}
if(i%3!=0)
{
    cout<<"*ERRORE , CODONI INCOMPLETI*\n";
    fout<<"*ERRORE , CODONI INCOMPLETI*";
}
else if(errore)
{
    cout<<"*ERRORE , CARATTERI NON VALIDI*\n";
    fout<<"*ERRORE , CARATTERI NON VALIDI*\n";
}
else
{
    if(i==0)
    {
        cout<<"ATTENZIONE , INPUT MANCANTE\n";
        fout<<"ATTENZIONE , INPUT MANCANTE\n";
    }
    else
    {
        cout<<"INPUT VALIDO\n";
    }
}
}

```

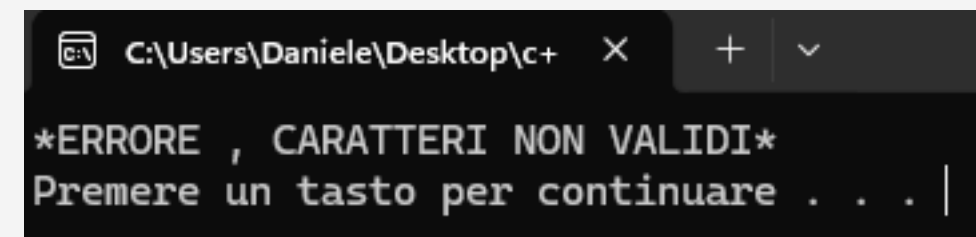
Il codice effettua una prima lettura in cui controlla la validità dei codoni dal punto di vista sintattico, i quali sono composti da triplette di amminoacidi. Nel caso in cui ciò non avvenga (o se nel file di input siano presenti caratteri non validi), viene notificato all'utente



```

C:\Users\Daniele\Desktop\c+ >
*ERRORE , CODONI INCOMPLETI*
Premere un tasto per continuare . . .

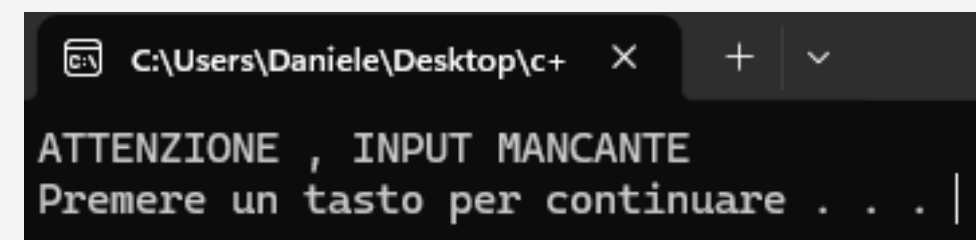
```



```

C:\Users\Daniele\Desktop\c+ >
*ERRORE , CARATTERI NON VALIDI*
Premere un tasto per continuare . . . |

```



```

C:\Users\Daniele\Desktop\c+ >
ATTENZIONE , INPUT MANCANTE
Premere un tasto per continuare . . . |

```

```

if(s==0)
{
    if(codone[x][y][z]=='M')
    {
        fout << 'M';
        s=1;
    }
    else
    {
        fout << "*CODONE D'INIZIO MANCANTE*";
        s=-1;
    }
}
else if (s==1)
{
    if(codone[x][y][z]=='*')
    {
        s=2;
    }
    else
    {
        fout << " " << codone[x][y][z];
    }
}

i = -1;

```

```

if(s==1)
{
    fout.close();
    fout.open("proteine.txt", ios::out | ios::trunc);
    fout<<"*CODONE DI STOP ASSENTE*";
}

```

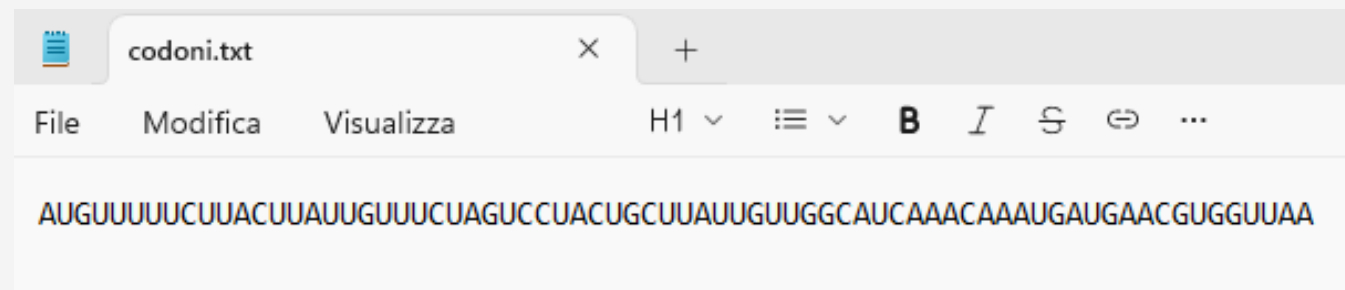
Il codice si occupa quindi della traduzione, segnalando all'utente eventuali errori, come codoni incompleti, assenza del codone d'inizio e mancanza dei codoni di stop.

Il codice sfrutta il concetto di macchina a stati finiti secondo questo schema :



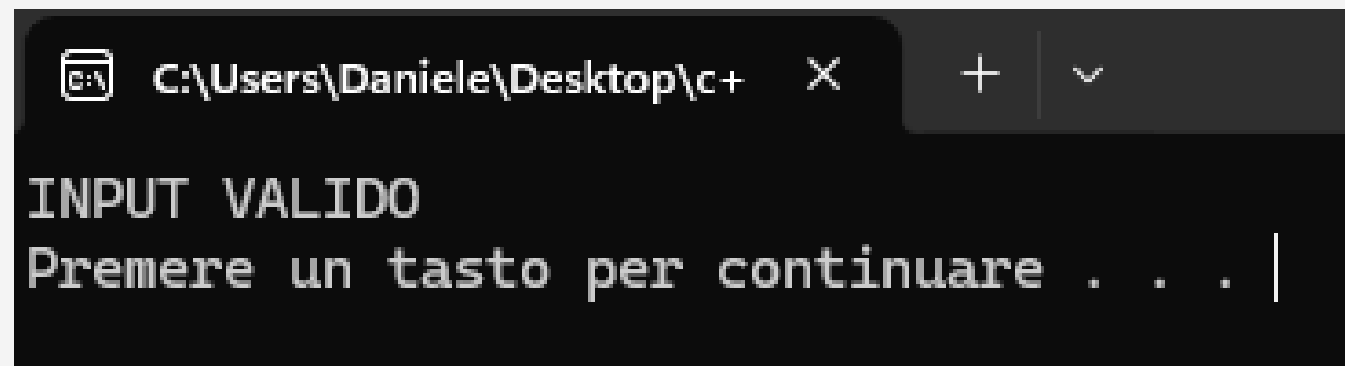
Se l'automa è ancora nella fase 1, ma la traduzione è completata, allora è assente il codone di stop. Tuttavia, se il primo codone tradotto risulta diverso dalla metionina, la traduzione viene interrotta immediatamente.

Esempio di funzionamento



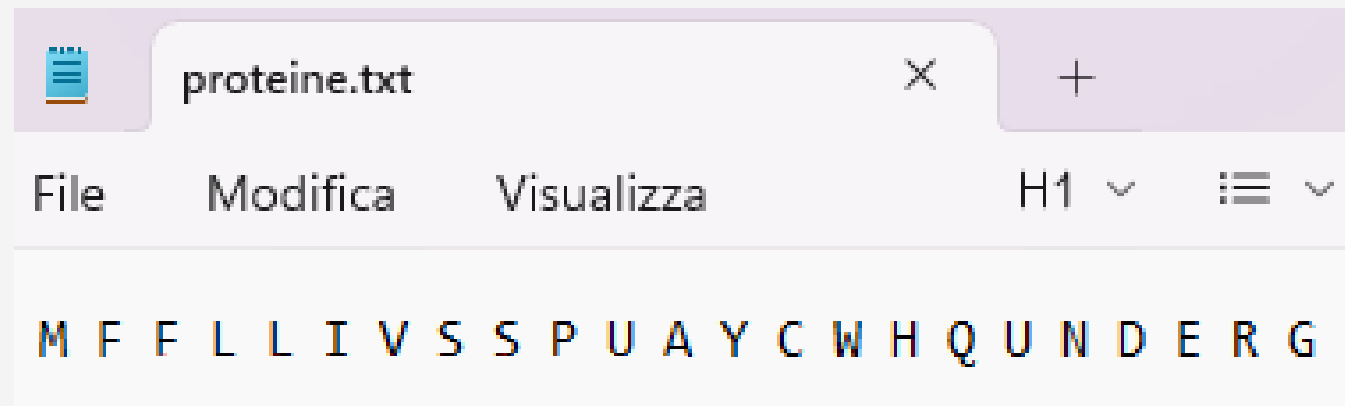
```
codoni.txt
File Modifica Visualizza H1  B I S ...
AUGUUUUUCUUAUUUUUUUAGUCCUACUGCUUAUUUGGUUGCAUCAAACAAUGAUGAACGUGGUUAA
```

Nel file codoni.txt viene inserita una serie di codoni



```
C:\Users\Daniele\Desktop\c+
INPUT VALIDO
Premere un tasto per continuare . . . |
```

Viene avviato il programma che avverte l'utente della validità dell' input



```
proteine.txt
File Modifica Visualizza H1  ...
M F F L L I V S S P U A Y C W H Q U N D E R G
```

Nel file proteine.txt viene prodotta la serie di amminoacidi corrispondenti